

# CSE234 Week 6: C Preprocessor, Multi-file Projects, and Exception Handling

This week, we'll look at a variety of topics that aren't long enough on their own to dedicate an entire week to. First, we'll look at C preprocessor directives. These directives form a meta-language embedded within C and C++ code, which is interpreted by the compiler before the code is compiled. We will also discuss how to break a project up into multiple files. Finally, we'll discuss exception handling in C++.

## C Preprocessor

The C preprocessor is a tool that runs before the compiler runs on C-family languages, including C++. C preprocessor directives are a meta-language for providing instructions to the preprocessor. Studies have shown that preprocessor directives make code harder to understand, but there are some tasks which can only be accomplished using the preprocessor. Preprocessor directives start with the pound sign `#`. Unlike C++ code, where statements must be ended by semicolons, preprocessor statements are ended by new lines.

## Include

The first preprocessor directive we'll discuss is include. You've been using this since you started using C++. It tells the preprocessor to find the specified file and include its code in the file. For example,

`#include <iostream>` tells the preprocessor to find the `iostream`

file and paste its code into the file that contains the include directive. Files within angle brackets `<>` are searched for in the system header file directories. The actual location varies by operating system. Files in double-quotes are searched for in the current directory. This allows files

that you write to be included as well.

## Define

The define directive is used to define macros. We'll consider three categories of macros. First, there are macros that have no value. These are just placeholders to be used by the preprocessor. A value-less macro is created like the following:

```
#define MACRO
#define ANOTHER_MACRO

ANOTHER_MACRO MACRO int x;
```

Typically, these are used as flags for the preprocessor, and do not appear in the code at all after the preprocessor runs. For example, after the preprocessor runs, the above code will become:

```
int x;
```

Next, there are macros that have values. When the preprocessor runs, it will replace any instance of the macro name with the exact text defined for it:

```
#define ONE_PLUS_ONE 1 + 1
int x = ONE_PLUS_ONE;
```

When the preprocessor runs, this code will become:

```
int x = 1 + 1;
```

Macros only perform text replacement. The `1 + 1` is not evaluated by the preprocessor. The "value" of a macro can be any text, which is simply inserted into the code anywhere macro name occurs.

The final macro variety is function-like macros. Function-like macros have a parameter list, and can be "invoked" like a function in the code. The body of the macro follows the parameter list. Since macros are ended by new lines, the body of the macro needs to be placed on a single line. A trick to get around this is to escape the new line by placing

a backslash `\` at the end of the line. Anywhere a parameter occurs in the macro body, it is replaced by the actual value passed to the macro. The body of the macro, after the replacement, is then inserted into the code. Like the "value" macros, no evaluation is performed by the preprocessor.

```
#define ADD(a, b) (a + b)
int z = ADD(1, 2);

int x = 3;
int y = 5;
z = ADD(x, y);

#define MULTI_LINE_MACRO(x) \
    std::cout << x << "\n";

// there is a semicolon in the macro so
// we don't have to put it here because
// it will be added by the preprocessor
// with the rest of the macro body
// when the macro is replaced
MULTI_LINE_MACRO("using a multiline macro")
```

After running the preprocessor, the code will be the following:

```
int z = (1 + 2);

int x = 3;
int y = 5;
z = (x + y);

std::cout << "using a multiline macro" << "\n";
```

The process of replacing macros is called "expansion". Things can get weird with macro expansion, and it is beyond the scope of this course. Again, preprocessor statements (and especially macros) are best avoided whenever possible. However, you will encounter them in real world code, so it is important to have some understanding of what this code does.

Macros can also be undefined using the `#undef` directive:

```
// defines the macro MACRO
#define MACRO
// undefines MACRO
#undef MACRO
```

## Conditional Compilation

The preprocessor can be used to conditionally compile code. This is where the value-less macros are often used. This is one of the places where the preprocessor and macros must be used. Different code may need to be compiled depending on the operating system. The C preprocessor offers `#if`, `#elif`, `#else`, `#ifdef`, and `#ifndef` for conditionals. All preprocessor conditional directives must be ended by a `#endif`. `#ifdef` is a shorthand to check if a symbol (usually a macro) is defined, and `#ifndef` is a shorthand to check if

For example, sometimes C++ programs have to interact with the system calls in the operating system specific headers. On Linux, there is the `unistd.h` library. On Windows, there is `windows.h`. Additionally, the functions within these libraries are different. Therefore, different code must be compiled depending on the operating system. Most Windows compilers define the `_WIN32` and `_WIN64` macros so that you can check if the code is being compiled on Windows. On Linux compilers, there is the `__unix__` macro. Apple Clang provides the `__APPLE__` macro.

```
// checking if we are on windows
#ifdef _WIN32
    #include <windows.h>
// mac and linux can use the same header most of the time
#else
    #include <unistd.h>
#endif
#include <iostream>

int main() {
    std::cout << "Sleeping for 2 seconds\n";

    #ifdef _WIN32
        // the windows version has a capital 'S'
        // and takes milliseconds as its argument
        Sleep(2000);
    #else
        // the Unix-like version is lowercase
```

```
        // and takes seconds as its argument
        sleep(2);
    #endif

    std::cout << "Done sleeping\n";

    return 0;
}
```

## Pragma

The `#pragma` directive is not a part of the C++ standard, but is defined by many compilers. It allows additional information to be passed to the compiler. It can be used, for example, to disable warnings. `#pragma once` tells the preprocessor to ensure that the code in a file is only included once.

## Splitting Code into Multiple Files

Up until this point you've primarily written all of your code in a single file. However, as the size of projects grow, this becomes untenable. Projects with thousands to millions of lines of code need to be separated into multiple files, or even into many files in multiple directories.

Let's look at the example code below. (For brevity, comments explaining the code are excluded.) This code is for a basic bank account simulation. It declares a `BankAccount` class, a main function that uses it, and the implementation of the class. *BankAccountProgram.cpp*

```
#include <string>
#include <iostream>

class BankAccount {
public:
    BankAccount();
    BankAccount(double startingBalance, std::string name);

    std::string getAccountHolder() const;
    double getCurrentBalance() const;

    void transferOwnership(std::string name);
    void deposit(double amount);
    bool withdraw(double amount);
private:
```

```

    double balance;
    std::string owner;
};

std::ostream& operator<<(
    std::ostream& out,
    const BankAccount& account
);

int main() {
    BankAccount account(1000, "John Doe");
    std::cout << account << std::endl;

    account.deposit(500);
    std::cout << account << std::endl;

    account.transferOwnership("Jane Doe");
    std::cout << "New owner: "
        << account.getAccountHolder() << std::endl;

    if (account.withdraw(1000)) {
        std::cout << "Successfully withdrew $1000" << std::endl;
        std::cout << "New balance: $"
            << account.getCurrentBalance() << std::endl;
    }
    else {
        std::cout << "Insufficient funds to withdraw $1000"
            << std::endl;
        std::cout << "You are $"
            << 1000 - account.getCurrentBalance()
            << " short" << std::endl;
    }

    return 0;
}

BankAccount::BankAccount() {
    balance = 0;
    owner = "";
}

BankAccount::BankAccount(double startingBalance, std::string name) {
    balance = startingBalance;
    owner = name;
}

std::string BankAccount::getAccountHolder() const {
    return owner;
}

double BankAccount::getCurrentBalance() const {

```

```

        return balance;
    }

    void BankAccount::transferOwnership(std::string name) {
        owner = name;
    }

    void BankAccount::deposit(double amount) {
        if (amount < 0) {
            std::cerr
                << "Invalid deposit - should not be negative! Abort..."
                << std::endl;
            exit(1);
        }

        balance += amount;
    }

    bool BankAccount::withdraw(double amount) {
        if (amount < 0) {
            std::cerr
                << "Invalid withdrawl - should not be negative! Abort..."
                << std::endl;
            exit(2);
        }

        if (amount <= balance) {
            balance -= amount;
            return true;
        }
        else return false;
    }

    std::ostream& operator<<(
        std::ostream& out,
        const BankAccount& account
    ) {
        out << "Balance: " << account.getCurrentBalance() << std::endl;
        out << "Owner:  " << account.getAccountHolder() << std::endl;
        return out;
    }

```

As you can see, we have a class declaration for `BankAccount` at the top of the file. We also have a free function to overload the insertion operator (for printing). I could have made this a friend, but since I don't need direct access to any private members (I have getters for both of them), I chose to avoid making it a friend. In general, I recommend avoiding friend functions whenever possible. Following this, we have

the `main` function, which plays around with a `BankAccount` object called `account`. Then we have the implementation of the `BankAccount` methods.

In C++, it is a common pattern for each class to have its own header (.hpp) and implementation (.cpp) file. The `main` function should also be in its own file (.cpp). Since the free function for `operator<<` is strongly coupled with the `BankAccount` class, I'll include its declaration and implementation with the class `BankAccount`. This suggests that we need three files for our program: one for the `main` function, one for the declaration of `BankAccount`, and one for the implementation of `BankAccount`. This refactoring is shown below.

*BankAccount.hpp*



```

#ifndef BANKACCOUNT_HPP
#define BANKACCOUNT_HPP

#include <string>
#include <iostream>

#include <string>
#include <iostream>

class BankAccount {
public:
    BankAccount();
    BankAccount(double startingBalance, std::string name);

    std::string getAccountHolder() const;
    double getCurrentBalance() const;

    void transferOwnership(std::string name);
    void deposit(double amount);
    bool withdraw(double amount);
private:
    double balance;
    std::string owner;
};

std::ostream& operator<< (
    std::ostream& out,
    const BankAccount& account
);

#endif

```

### *BankAccount.cpp*

```

#include "BankAccount.hpp"

BankAccount::BankAccount() {
    balance = 0;
    owner = "";
}

BankAccount::BankAccount(double startingBalance, std::string name) {
    balance = startingBalance;
    owner = name;
}

std::string BankAccount::getAccountHolder() const {
    return owner;
}

```

```

}

double BankAccount::getCurrentBalance() const {
    return balance;
}

void BankAccount::transferOwnership(std::string name) {
    owner = name;
}

void BankAccount::deposit(double amount) {
    if (amount < 0) {
        std::cerr
            << "Invalid deposit - should not be negative! Abort..."
            << std::endl;
        exit(1);
    }

    balance += amount;
}

bool BankAccount::withdraw(double amount) {
    if (amount < 0) {
        std::cerr
            << "Invalid withdrawl - should not be negative! Abort..."
            << std::endl;
        exit(2);
    }

    if (amount <= balance) {
        balance -= amount;
        return true;
    }
    else return false;
}

std::ostream& operator<<(
    std::ostream& out,
    const BankAccount& account
) {
    out << "Balance: " << account.getCurrentBalance() << std::endl;
    out << "Owner:  " << account.getAccountHolder() << std::endl;
    return out;
}

```

*main.cpp*

```

#include <iostream>
#include "BankAccount.hpp"

```

```

int main() {
    BankAccount account(1000, "John Doe");
    std::cout << account << std::endl;

    account.deposit(500);
    std::cout << account << std::endl;

    account.transferOwnership("Jane Doe");
    std::cout << "New owner: "
        << account.getAccountHolder() << std::endl;

    if (account.withdraw(1000)) {
        std::cout << "Successfully withdrew $1000" << std::endl;
        std::cout << "New balance: $"
            << account.getCurrentBalance() << std::endl;
    }
    else {
        std::cout << "Insufficient funds to withdraw $1000"
            << std::endl;
        std::cout << "You are $"
            << 1000 - account.getCurrentBalance()
            << " short" << std::endl;
    }

    return 0;
}

```

The header file was named using the name of the class, `BankAccount`, followed by `.hpp`. Since we are putting one class in each file, it makes sense to name them this way. We have a corresponding implementation file, `BankAccount.cpp`. The `main` function is in a file `main.cpp`. Sometimes, this file may be named after the overall program. Since this is a very basic example program, calling it `main.cpp` or `driver.cpp` is acceptable.

In the header file, we have some preprocessor directives. The file begins with `#ifndef BANKACCOUNT_HPP` and `#define BANKACCOUNT_HPP`, and ends with `#endif`. This pattern is referred to as "include guards". The `#ifndef BANKACCOUNT_HPP` checks if the macro `BANKACCOUNT_HPP` is not defined. The code between the `#ifndef` and `#endif` is only compiled if that macro is not defined. Then, the next line immediately defines the `BANKACCOUNT_HPP` macro. This guards against the same code being compiled twice in the same program. `BANKACCOUNT_HPP` is

defined the first time the code is compiled, so only the first instance gets compiled.

In the `.cpp` source code files, you will see that we have `#include "BankAccount.hpp"`. Typically, when we have included files, we've used angle brackets `<>`. The angle brackets tell the compiler to search for standard library system headers that were included with the compiler or installed libraries. However, the double quotes tell the compiler to instead search the current directory first to find the included file. Also, note that we only ever include the `.hpp` file. You should never include a `.cpp` file. Because the header file has to be included in both files, the two source files are compiled separately, then combined in a process called linking at the end of the compilation process.

Otherwise, we've pretty much just copy-pasted the code for the `BankAccount` class declaration and `operator<<` function declaration into `BankAccount.hpp`. We copied the `main` function into `main.cpp`. And all the other function definitions were copied into `BankAccount.cpp`.

IDEs like Visual Studio and CLion handle compiling and linking separate files for you. Build tools like CMake (which we'll discuss towards the end of the course) also simplify compilation. To manually compile a three file project, the following commands are used (assuming the `g++` compiler is being used):

```
# compile the two source files separately with the -c flag
g++ -c BankAccount.cpp -o BankAccount.o
g++ -c main.cpp -o main.o
# header files are not compiled
# they are included in the .cpp files which are compiled

# now, link the files
g++ BankAccount.o main.o -o BankAccountProgram
# run the program
./BankAccountProgram
```

## Exception Handling

Exceptions are data types that store information about a run-time error in a program. Exception handling allows programs to recover gracefully from error conditions. There are four aspects of error handling to

consider: exception data types, try blocks, catch blocks, and throw statements. Exception data types are classes (or structs) which contain information about the exception. Try blocks specify a block of code that will pass exceptions to a catch block. The catch block specifies what the program does when the exception occurs. A try block can have multiple catch blocks to handle different exceptions. A throw statement is used to throw an exception. Throwing an exception will immediately exit the current function and pass the exception to the calling function. If an exception is unhandled, the program will crash.

## Exception Data Types and Throw Statements

The C++ standard library provides some exception types in the `exception` library. The base `std::exception` class serves as the base exception type, which the standard library exceptions inherit from. We'll talk about inheritance in detail during the last third of the course, but for now a basic introduction is all we need. Classes in C++ can inherit from other classes. The class being inherited from is often called the base, or parent, class. Classes that do the inheriting are called derived, or child, classes. C++ supports public, protected, and private inheritance. For now, we'll only consider public inheritance. This means that the derived classes have access to all public and protected member functions and fields of the base class. The main thing of interest in the `std::exception` class is the function `virtual const char* what();`. When called for an instance of the `std::exception` class, it returns a C string that has a compiler defined error message. Derived classes create their own implementation of the `what()` function that describe the specific type of exception.

Some examples of standard library derived exceptions include `std::runtime_error`, `std::logic_error`, and `std::bad_cast`. Some of these have further derived exception classes that are more specific. Derivations of `std::runtime_error` include `std::range_error`, `std::underflow_error`, and `std::overflow_error`. `std::logic_error` has derivations such as `std::out_of_range` and `std::invalid_argument`. These classes and their derivations implement the `what()` function and have constructors that are passed an error message. The error message can be either a C string or an `std::string`. `what()` will return whatever

error message was passed. Exception objects are typically created within throw statements, although it's not a requirement. Throw statements are similar to return statements, except they use the `throw` keyword instead of `return`. `throw` can throw any expression. Below is an example using the `std::out_of_range` exception. The function returns a character from a C string. If the index passed to the function is out of range, it throws an out of range error.

```
// needed for strlen
#include <cstring>
// needed for the std::out_of_range exception type
#include <exception>
#include <format>
#include <string>

char getChar(const char* str, int index) {
    int length = strlen(str);
    if (index > length - 1 || index < 0) {
        std::string what = std::format(
            "Invalid index provided. {} is not in the range 0 to {}",
            index, length
        );
        throw std::out_of_range(what);
    }
}
```

We can also create our own exception types. While we do not need to inherit from the `std::exception` class, doing so will allow catch blocks that handle `std::exception` references to also handle our own exceptions. Below is a basic exception:

```
#include <exception>
#include <string>

// to inherit from exception, we put a colon after
// the name of the exception class. we will
// specify that we are using public inheritance
class CustomException: public std::exception {
public:
    CustomException() : msg("CustomException") {}

    CustomException(std::string errorMessage)
        : msg(errorMessage) {}

    CustomException(const char* errorMessage)
        : msg(errorMessage) {}
}
```

```

// = default lets tells the compiler to implement the destructor
// we can do this since there are no fields in this class using
// pointers the override tells the compiler to use the derived
// class destructor instead of std::exception's
~CustomException() override = default;

// once again, override tells the compiler to use this version of
// what() instead of std::exception's version
const char* what() override { return msg.c_str(); }
private:
    std::string msg;
}

```

Again, we'll discuss the mechanisms of inheritance in more detail later in the course. For now, just follow this pattern for creating custom exceptions.

## Try and Catch Blocks

So now that we can create and throw exceptions, we need to handle them. To make it possible to catch an exception, we need to put the code that could throw an exception within a try block. A try block consists of the keyword `try` followed by a code block. Additionally, a try block must have at least one catch block. A catch block consists of the keyword `catch`, a set of parentheses that declares the type of exception the catch will handle, and a block that contains the statements that are executed when the exception is caught. Assume that the code below is in the `main()` function for the program with the `getChar()` function.

```

try {
    const char* str = "hello world";
    int length = strlen(str);
    // this will trigger an out_of_range exception
    // since <= is used instead of <
    for (int i = 0; i <= length; ++i) {
        std::cout << getChar(str, i) << "\n";
    }
}
// first catch block catches only out_of_range exceptions
// exceptions should be caught by reference
catch (std::out_of_range &ex) {
    std::cerr << ex.what() << "\n";
}
// handle any other exception that is derived from std::exception

```

```

catch (std::exception &ex) {
    std::cerr << "Unexpected exception occurred\n";
    std::cerr << ex.what() << "\n";
}
// handle any other exception. the ... parameter is used
// this must be the last catch for a try block, if it is used
catch (...) {
    std::cerr << "Unknown exception occurred\n";
}

```

While try-catch statements are typically used for exceptions, technically anything can be thrown:

```

try {
    std::cout << "Throwing an int\n";
    throw 21;
}
catch (int& e) {
    std::cerr << "Caught " << e << "\n";
}

```

This will print:

```

Throwing an int
Caught 21

```

## Example

We can modify the above `BankAccount` class to throw exceptions rather than exit when invalid input is passed to the `deposit()` and `withdraw()` methods:

*BankAccount.hpp*

```

// add exception header
#include <exception>
// ...
// define exception class
class InvalidAmountError: std::exception {
public:
    InvalidAmountError() {}

    InvalidAmountError(std::string errorMessage)
        : msg(errorMessage) {}

    InvalidAmountError(const char* errorMessage)

```



```

        : msg(errorMessage) {}

~InvalidAmountError() override = default;

const char* what() override { return msg.c_str(); }
private:
    std::string msg;
};
// ...

```

### *BankAccount.cpp*

```

// ...
void BankAccount::deposit(double amount) {
    if (amount < 0) {
        throw InvalidAmountError(
            "Cannot deposit negative cash value");
    }

    balance += amount;
}

bool BankAccount::withdraw(double amount) {
    if (amount < 0) {
        throw InvalidAmountError(
            "Cannot withdraw negative cash value");
    }

    if (amount <= balance) {
        balance -= amount;
        return true;
    }
    else return false;
}
// ...

```

And, the main function can now be modified to catch the exceptions:

### *main.cpp*

```

#include <iostream>
#include "BankAccount.hpp"

int main() {
    BankAccount account(1000, "John Doe");
    std::cout << account << std::endl;

    account.deposit(500);
    std::cout << account << std::endl;
}

```

```

account.transferOwnership("Jane Doe");
std::cout << "New owner: "
    << account.getAccountHolder() << std::endl;

try {
    if (account.withdraw(1000)) {
        std::cout << "Successfully withdrew $1000"
            << std::endl;
        std::cout << "New balance: $"
            << account.getCurrentBalance() << std::endl;
    }
    else {
        std::cout << "Insufficient funds to withdraw $1000"
            << std::endl;
        std::cout << "You are $"
            << 1000 - account.getCurrentBalance()
            << " short" << std::endl;
    }
}
catch (InvalidAccountError& ex) {
    std::cerr << ex.what() << "\n";
}

return 0;
}

```

## Conclusion

This week we discussed the C preprocessor, splitting programs into multiple files, and exception handling. The preprocessor is run before the C++ code is compiled. Preprocessor directives specify what the preprocessor needs to do. Writing larger programs requires splitting the code into multiple files. Header files (.hpp) are used to store function and class declarations. Source code (.cpp) files are used to implement functions and classes. Only header files are included using include statements. Do not include implementation files. Instead, source code files are compiled separately, then linked together at the end of the compilation process. Exceptions are data types that store information about errors. Try-catch blocks are used to catch and handle exceptions. Exceptions can be triggered using throw statements.